Learning Tournament Slots and Pattern Skillsets of osu!mania Charts from Expert Mappools and Crowd-Sourced Tags

Komi
j.ruza@udc.es

Abstract

Players and tournament organisers describe osu!mania charts in a shared vocabulary of *slots* (rice, long note, hybrid, scroll speed) and *skillsets* (jumpstream, chordjack, LN release, ...). Beatmap Nominators, who triage far more ranking requests than they can play, would benefit from an automatic description in these terms. We build one from public data only. The mappools of 115 official tournaments label the slot of 4 029 charts, and crowd-voted user tags on all 30 732 ranked osu!mania difficulties label skillsets, which we treat as positive–unlabelled data with a strict and a relaxed threshold. A chart is represented twice: as one interpretable vector (71 statistics, their distribution over 16-second windows, a bag of row transitions, *reading* features that describe what scroll-speed changes do to the notes on screen, and MinaCalc skillset ratings), and as a sequence of note events. A gradient-boosted classifier on the statistics predicts the tournament slot with 98.1% accuracy, against 88.5% for slotting rules. For 11 reviewer categories and 20 user tags we average four model families: boosted heads, a multi-task MLP, and two event encoders (dilated convolutions and a transformer over 4-second windows, with attention-based multiple-instance heads) that are pretrained with masked-event and masked-window objectives and distilled from per-fold teachers on the 26,536 charts without a strict label. Under cross-validation grouped by song over all 36,509 charts, in which no model sees any chart of a test song, the ensemble reaches a mean average precision of 0.768 on the categories and 0.636 on the tags, against 0.719 and 0.588 for a boosted head on statistics and windows ($+0.049$ and $+0.047$; 95% song-bootstrap intervals $[+0.036, +0.060]$ and $[+0.040, +0.055]$).

1 Introduction

osu! is a free rhythm game with a large community of chart authors (*mappers*). Its osu!mania mode is a vertical-scrolling key game in the tradition of Beatmania and StepMania: notes fall down K columns ($K \in \{4, 7\}$ in most play) and must be hit, or for *long notes* (LNs) held and released, in time with the music. A chart (*difficulty*) enters the ranked section only after two Beatmap Nominators (BNs) review and nominate it. BNs receive far more requests than they can play, and choose what to check largely by what kind of chart it is.

The community has a precise, if informal, vocabulary for that. Tournament mappools group charts into *slots*, of which four are near-universal: *rice* (RC; tap notes), *long note* (LN), *hybrid* (HB; a mix of both) and *scroll speed* (SV; charts whose difficulty comes from changing scroll velocity). Within a slot, players speak of *skillsets*: jumpstream, handstream, chordjack, speed jacks, technical rhythm, stamina, LN release, LN coordination, inverse LNs, reading and memorising scroll changes, and so on. A reviewer who knows that a request is "a 4K hybrid with dense inverse LNs and technical rhythm" can route or prioritise it without opening it.

Producing that description automatically raises three problems. *Labels*: there is no annotated dataset. Tournament organisers slot charts carefully but only at the slot level; osu!(lazer) lets players vote *user tags* on charts they have passed, which gives skillset labels, but only for ranked charts with an audience, and only positively: a tag nobody voted for is not evidence that the pattern is absent. *Representation*: charts vary in key count, length, tempo and note count by orders of magnitude; the skillsets of interest are defined on short local configurations (a jack is the same column hit twice in quick succession); and some of them, such as an LN release section in a rice chart, occupy only part of the chart. *Evaluation*: tournament pools contain rate-altered copies of one song and a ranked beatmapset contains several difficulties of the same song, so random splits leak, and unlabelled charts used for pretraining or distillation can leak too.

This paper describes a system that addresses each in turn (Figure 1). Our contributions are:

- Weakly supervised datasets built from public pages only: tournament slots for 4 029 charts, and strict and relaxed skillset labels for 11 reviewer categories and 20 user tags, derived from user-tag votes and tournament subslots (Section 3).
- Two complementary chart representations: an interpretable vector of chart statistics, window statistics, row transitions, reading features and MinaCalc ratings, and a tokenisation of the chart into note events (Section 4).
- A slot classifier, and an ensemble of four model families for the categories and tags, in which neural students learn from unlabelled charts through per-fold teachers and self-supervised objectives without ever seeing a test song (Section 5).
- An evaluation grouped by song over the whole corpus, with paired song-bootstrap intervals, against rule baselines and a boosted reference on identical charts (Section 7).

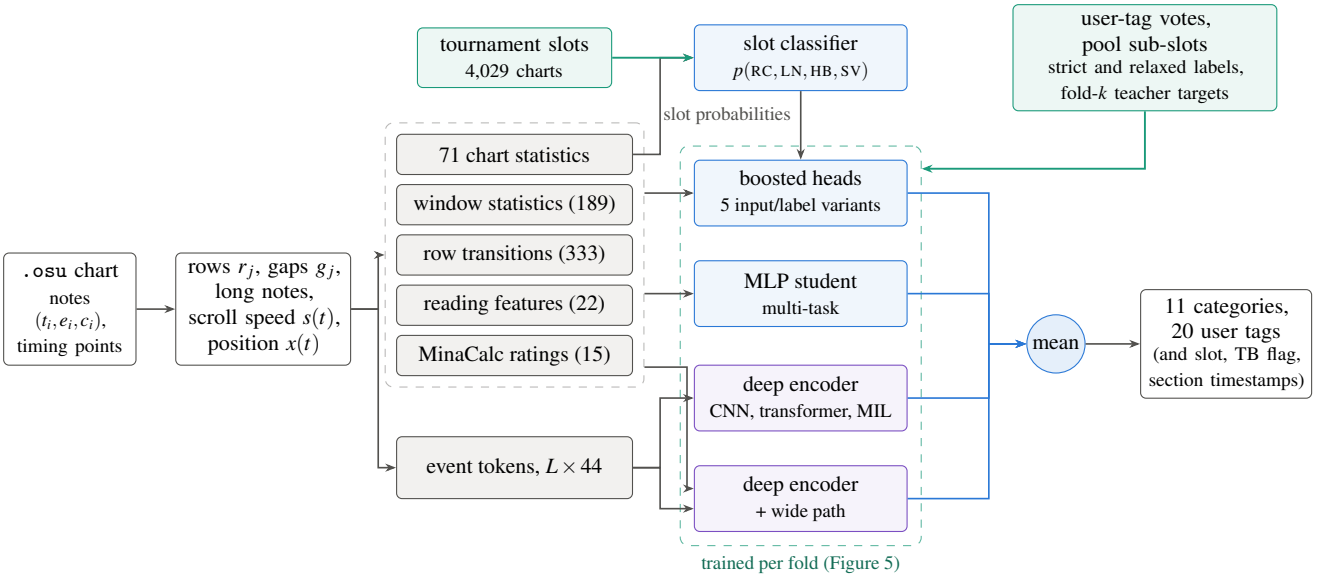


Figure 1: The system. A chart is parsed into rows, long notes and a scroll profile, and represented both as one vector of interpretable features per chart and as a sequence of note events. A boosted classifier trained on tournament slots predicts the slot. Four model families predict the reviewer categories and user tags and are averaged: boosted heads and an MLP on the feature vector, and two event encoders, one of which also sees the feature vector through a wide path. Green: supervision.

2 Background and related work

osu!mania charts. A chart is a list of hit objects (t_i, e_i, c_i) with start time t_i, end time $e_i \geq t_i$ ($e_i > t_i$ for an LN) and column $c_i \in \{0, \ldots, K-1\}$, plus a list of *timing points*. Uninherited ("red") points set the beat length and hence the BPM; inherited ("green") points set a slider-velocity (SV) multiplier that, in osu!mania, scales how fast notes scroll. Together they determine an effective scroll speed that may differ greatly from the song's tempo; SV charts exploit this.

Difficulty and pattern analysis. osu!'s own star rating is a hand-designed strain model that outputs a single number. Etterna's MinaCalc rates charts per skillset (stream, jumpstream, handstream, stamina, jackspeed, chordjack, technical) with hand-tuned pattern detectors [13]; we use its ratings as features. Community tools for osu! are also rule-based: rosu-pattern-detector contributors [31] detect mania jacks and jump/hand streams, while SimpleBeatmapAnalyzer [28] and osu-finder [26] classify osu!standard maps by spacing or strain ratios. None learns from labels or outputs tournament slots. Our rule baselines (Section 5.1) are in the spirit of these tools.

Machine learning on rhythm-game charts. Most learned work generates charts from audio: Dance Dance Convolution [10] and its successors [1, 38], and TaikoNation, which targets human-like patterning [18]. Franks et al. [14] estimate StepMania difficulty levels with ordinal regression, and Liu and Donahue [24] use osu! timing as beat-tracking labels. To our knowledge no prior work classifies charts into tournament slots or predicts crowd skillset tags.

Learning from tags and positive-only labels. Predicting user tags is analogous to music auto-tagging [4, 23], where social tags are sparse and under-reported [16]. A missing tag is an unlabelled example rather than a negative one, the setting of positive–unlabelled (PU) learning [2, 12] and of multi-label learning with missing labels [39]. We abstain on insufficiently annotated charts and down-weight weakly annotated ones; combining several noisy label sources by rules is a form of weak supervision [29].

Models. The boosted heads are gradient-boosted trees [15] in the histogram formulation of LightGBM [22] as implemented in scikit-learn [27], which remain strong on tabular data [17]; feeding rule scores to them is stacking [35]. The neural students are trained on teacher probabilities where labels are missing, which is knowledge distillation [19] used for self-training on unlabelled data [37]. The event encoder combines gated [5] dilated [33] convolutions with a transformer [34], is pretrained by masked prediction [7], and reads out each label with attention-based multiple-instance pooling [9, 20], since a chart is positive for a skillset when some of its sections are. Its wide path follows wide-and-deep models [3], and the final prediction is a plain ensemble average [8]. Group-aware cross-validation avoids the leakage that related samples cause [21, 30].

3 Data

Everything is collected without an osu! API key: the osu! wiki is a public Git repository, chart files are served at `osu.ppy.sh/osu/{id}`, and the public beatmapset page embeds the same JSON the API returns, including each difficulty's user tags with vote counts.

3.1 Tournament mappools (slot labels)

The osu! wiki documents the mappools of officially supported tournaments. We parsed 9 963 pool entries (8 454 distinct charts) from 115 osu!mania tournaments in 42 series (e.g. MWC, MCNC, 4DM, SOFT, o!mLN, JHC) and downloaded every chart. A normaliser maps raw slot names ("LN",

"Long Note", "Long Notes") onto $\mathscr{Y} = \{\text{RC}, \text{LN}, \text{HB}, \text{SV}\}$; tournaments that are wholly one category inherit it (all of o!mLN is LN, all of SVSE is SV, and the jack-and-chord tournament JHC is RC). FreeMod, NoMod, Tiebreaker and Wildcard slots carry no pattern information and are left unlabelled. A chart in several pools receives the majority label and exact ties are discarded. After removing charts with fewer than 50 notes or fewer than 4 keys, 4 029 charts of 3 558 songs are labelled. Sub-slot names ("Dense Chordjack", "Coordination", "Memorisation", "Tiebreaker") are kept for the categories of Section 3.3.

3.2 User tags on ranked charts (skillset labels)

We listed all 8 673 ranked osu!mania beatmapsets and fetched the user tags of their 30 732 difficulties. Votes are sparse: 10 031 difficulties have at least one vote and 3 227 have eight or more. Let $v_{d,\tau}$ be the number of votes for tag τ on difficulty d and $V_d = \sum_\tau v_{d,\tau}$. The *strict* label, used for training and for all evaluation, is

$$y_{d,\tau} = \begin{cases} 1 & v_{d,\tau} \geq 2, \\ 0 & v_{d,\tau} = 0 \text{ and } V_d \geq 8, \\ \varnothing & \text{otherwise}, \end{cases} \tag{1}$$

where $\varnothing$ (unknown) removes the pair from that tag's training and evaluation sets: a single vote is often idiosyncratic, and a difficulty with few votes says little about the tags it lacks. The *relaxed* label adds, for training only and at weight $\frac{1}{2}$, the pairs that Equation (1) leaves unknown but that carry some evidence: $y = 1$ if $v_{d,\tau} = 1$, and $y = 0$ if $v_{d,\tau} = 0$ and $3 \leq V_d \leq 7$. We keep the 20 mania tags with at least 12 strict positives and 12 strict negatives (Table 4).

3.3 Reviewer categories

The BN who requested this tool defined the categories in which predictions are reported (multi-label): *Rice*: Chordjack, Streams, Jumpstream/Handstream, Technical, Mixed Rice, Stamina; *LN*: Release, Coordination, Density/Inverse; *SV*: Reading, Memo; *Hybrid*; and *TB* (tiebreaker-worthy). Hybrid is the slot of Section 3.1, and Mixed Rice is derived: a chart is Mixed Rice when skills from at least two of the families {Chordjack}, {Streams, Jumpstream/Handstream}, {Technical} fire. The other 11 categories have learned heads and draw labels from two sources, combined by "positive anywhere wins, otherwise negative anywhere, otherwise unknown":

- user tags, through a fixed many-to-one map (e.g. Jumpstream/Handstream ← `style/jumpstream`, `style/handstream`, `style/chordstream`, `style/quadstream`) with Equation (1) applied to the category's most-voted tag, and the relaxed rule likewise;
- tournament sub-slots (JHC's chordjack and quadstream slots, o!mLN's Coordination and Density, SVSE's Reading and Memorisation, and every pool's Tiebreaker). A chart in one sub-slot is a negative for the sibling sub-slots of the same tournament, because poolers slot by the dominant skill; for TB, every other slot-labelled pool chart is a negative.

Table 1: The chart corpus of the category and tag experiments. Every chart is assigned to one of five folds by its song; a model for fold k never sees a chart of a fold-k song.

Charts in the corpus	36 509
in a tournament pool	7 251
ranked, not pooled	29 258
Songs (fold groups)	11 376
Charts with ≥ 1 strict category label	9 559
Charts with ≥ 1 strict tag label	5 640
Charts with no strict label	26 536
Relaxed (chart, task) labels added	102 725
of which positive	11 519

Two mappings needed care. The mania tag `skillset/reading` denotes reading dense patterns rather than scroll-speed reading, so SV Reading uses the `sliders/*` tags. Memo uses `gimmick/memory`, but only on charts whose scroll speed actually varies, since the tag is also voted on storyboard gimmicks.

3.4 Corpus and folds

The category and tag experiments use every ranked osu!mania chart together with every tournament chart we could download, 36,509 charts in all (Table 1). Most of them have no strict label for any task; they are still useful, as inputs for self-supervised pretraining and as charts on which a teacher's predictions can be distilled. To make that safe, the five cross-validation folds are drawn by *song* over the whole corpus: the song key is the normalised artist and title with mapper and difficulty names stripped, so rate edits, re-uploads and the difficulties of a beatmapset stay together. A model for fold k never sees any chart of a fold-k song, labelled or not.

4 Chart representation

4.1 Rows, gaps and scroll position

Notes whose start times lie within 2 ms form a *row* r_j at time τ_j with column set C_j. Let $g_j = \tau_j - \tau_{j-1}$; a transition is *fast* if $g_j < 220$ ms, inside a pattern rather than a pause. The timing points define a piecewise-constant effective scroll speed

$$s(t) = \frac{\beta_{\text{main}}}{\beta(t)}\,\text{sv}(t), \tag{2}$$

where $\beta(t)$ is the active beat length, β_{main} the beat length covering most of the chart, and $\text{sv}(t) \in [0.01, 10]$ the active SV multiplier, reset to 1 at each red point. Its integral $x(t) = \int_0^t s(u)\,du$ is the *scroll position*: a note hit at time T is drawn at height $x(T) - x(t)$ above the judgement line at time t.

4.2 Chart statistics

71 statistics describe the chart as a whole, each a quantity a mappooler inspects (Figure 2a; full list in Appendix A). They are normalised by note count, row count or duration so that charts of different key counts and lengths are comparable, and K is itself a feature. **Meta and density** (15): keys, OD, HP, note count, duration, notes per second overall and per key,

percentiles of 2-second note rates, peak-to-median ratio, a stamina proxy (the share of windows within 80% of the 90th-percentile rate) and the variation of density. **Chords, jacks and streams** (22): shares of single, jump, hand and quad rows; the share of fast transitions with $C_j \cap C_{j-1} \neq \emptyset$ (*jack rows*), of jack rows between two chords (*chordjacks*) and of fast transitions with $C_j \cap C_{j-1} = \emptyset$ (*stream rows*), split into jumpstream and handstream by $|C_j|$; jack gaps and chains; trills; and anchors. **Rhythm** (14): the distribution of the snap of fast gaps relative to the local beat (1/2 to 1/16, triplets, other), the number of distinct gaps per 16 rows, the rate of large gap-ratio changes, the main BPM and the number of BPMs. **Long notes** (11): LN share ρ, LNs per second, LN lengths in ms and beats, hold coverage $\sum_i (e_i - t_i)/(K \cdot T)$, the inverse rate (an LN followed in its column by a note less than 0.3 beats after its release), the share of releases on no row (*off-grid*), and the mean number of columns held while a note is hit, divided by K (*coordination*). **Scroll speed** (9): the duration-weighted standard deviation $\sigma_{\log s}$ of $\log s$, the share of time with $|\log s| > 0.05$ and > 0.4, of near-stops ($s < 0.2$) and fast sections ($s > 2$), SV point and change rates, and the rate of SV changes within 1.5 s of a note.

4.3 Window statistics

Chart-level statistics dilute short passages: a 20-second LN release section in a three-minute rice chart barely moves the chart's LN share. We therefore recompute the statistics on windows of 16 s every 8 s that contain at least 30 notes, dropping the 8 statistics that are chart constants or only measure window length, and pool each of the remaining 63 over windows by its maximum, 75th percentile and standard deviation (189 features, Figure 2c). This is a multiple-instance representation that needs no learning: "somewhere in this chart there is X" becomes a high maximum.

4.4 Row transitions

The statistics summarise patterns as a few shares, but many skillsets differ in their transitions rather than their chord sizes: a jumptrill and a jumpstream have the same rows in a different order. We count every pair of consecutive rows by their exact column sets (Figure 2b). In 4K the key is the pair of 4-bit masks, and a transition and its mirror image are counted together (117 classes). In 7K each hand is handled separately: columns 0–2 are mirrored onto columns 4–6, so that a roll towards the thumb reads the same on both hands, the two hands share one 3-bit vocabulary (7×7 pairs), and transitions are taken between consecutive rows in which that hand plays; the thumb column's share of rows is one more feature. Each pair is bucketed as fast (< 150 ms) or slow (150 ms to 400 ms); rows further apart are not a pattern. Counts are divided by the number of transitions, which gives 333 features; other key counts get none.

4.5 Reading features

The scroll-speed statistics describe the speed values. What makes an SV chart hard to read or memorise is what those values do to the notes on screen, which the reading features measure from $x(t)$ (Figure 3). Let H be the screen height, expressed as the time a note is visible at the main BPM and $1.0\times$ SV (we take 600 ms, a typical scroll speed). A note hit at T becomes visible at the time $o(T)$ where $x(T) - x(o(T)) = H$, so its *visible time* is $T - o(T)$; at constant speed every note is visible for exactly H. Two consecutive rows are drawn Δx apart for a time gap Δt; at constant speed $\Delta x/\Delta t = 1$. From these we compute 22 features (Appendix B): percentiles of the visible time in units of H, and the shares of notes that appear late ($< 0.5H$, $< 0.25H$) or crawl ($> 2H$); the spread of $\log(\Delta x/\Delta t)$, its extremes, its jitter between consecutive rows, and the shares of *stacked* ($\Delta x/\Delta t < 0.2$) and *spread* (> 3) rows; the variation of the speed at the moments notes are hit, overall and within 2-second blocks; the rate of speed changes that happen while the next note is already on screen, which make visible notes jump, stop or rush; the rates of *teleports* (segments faster than $4\times$ and shorter than 80 ms) and *freezes* (slower than $0.1\times$ and longer than 150 ms) while a note is visible; and how many notes are on screen at once relative to constant scroll.

4.6 MinaCalc ratings

MinaCalc (Etterna's calculator, version 527) rates a chart on overall difficulty and seven skillsets [13]. We run it at $1.0\times$ on every 4K, 6K and 7K chart, with long notes counted as their heads as in Etterna, and keep the eight ratings and the seven skillset ratings divided by the overall rating, which say what a chart is about independently of how hard it is (15 features).

4.7 Event tokens

For the neural encoders a chart becomes a sequence of *events*, the moments at which at least one note starts or a long note ends (merged within 2 ms; Figure 4). Each event carries, per column, three bits: *head* (a note starts), *tail* (an LN is released) and *held* (an LN body passes through the event without starting or ending there). The columns of a K-key chart are centred in 10 slots, so 4K and 7K charts share weights, and mirroring the chart reverses the slots. Five continuous values follow: the log gaps to the previous and next event, the gap to the previous event in beats, the log effective scroll speed and the log beat length; then a one-hot snap of the previous gap (1/1, 1/2, 1/4, 1/8, 1/16, 1/3, 1/6, 1/12, other). An event is thus a vector of $10 \cdot 3 + 5 + 9 = 44$ values; the key count enters the model separately. Events are grouped into 4-second windows by their time from the first event.

5 Models

5.1 Rule baselines

Every learned component has a hand-written counterpart evaluated on identical charts. The slot rule assigns SV if $\min(1, \sigma_{\log s}/0.3) \cdot \min(1, \text{near-note SV rate}/0.5) \geq 0.5$, and otherwise the argmax of three LN-share scores: RC $= \text{clip}(1 - \rho/0.12)$, HB $= \text{clip}(1 - |\rho - 0.25|/0.15)$, LN $= \text{clip}((\rho - 0.3)/0.3)$. Each tag rule is a product of logistic squashes $\sigma((x-m)/w)$ of the one or two statistics that define the tag (e.g. jumpstream $= (1 - \sigma_\rho)\,\sigma($(jumpstream rows –

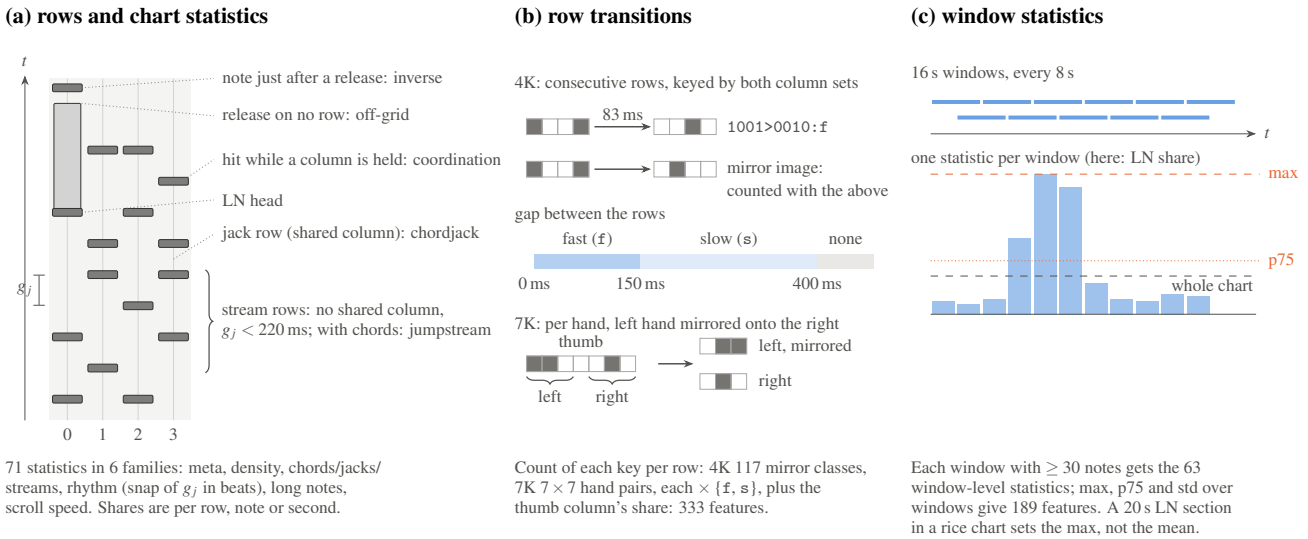


Figure 2: Chart-level features. (a) Notes are grouped into rows (time runs upward, as the notes fall); whether consecutive rows share a column, how far apart they are, and how long notes overlap other rows define the chart statistics. (b) Row transitions count each pair of consecutive row shapes, separately for fast and slow gaps; mirror images are merged in 4K, and 7K charts are split into hands. (c) Window statistics recompute the statistics on overlapping 16-second windows and keep their maximum, 75th percentile and standard deviation, so that a section-level skill is not averaged away.

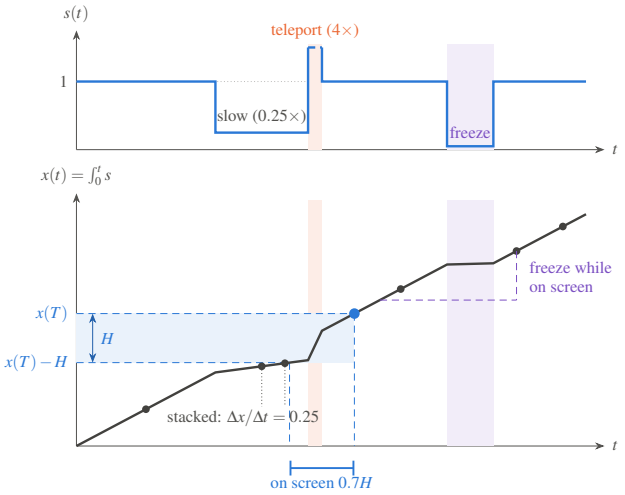


Figure 3: Reading features. Top: the effective scroll speed $s(t)$ of a chart with a slow section, a teleport and a freeze. Bottom: the scroll position $x(t)$ and the hit times of seven notes (dots). A note hit at T is on screen while $x(T) - x(t) \le H$: the highlighted note appears near the end of the slow section and is visible for only $0.7H$. The two notes in the slow section are drawn a quarter of their time gap apart (stacked), and the freeze happens while the second-to-last note is on screen.

$0.18)/0.04)$, where $1 - \sigma_\rho$ suppresses LN charts); category rules take the maximum over their tags' rules. The thresholds were set by inspecting a few dozen charts before any labels were collected.

5.2 Slot classifier

The slot model is a histogram gradient-boosting classifier [22, 27] on the 71 statistics with 300 iterations, learning rate 0.05, at most 15 leaves, ℓ_2 regularisation 1.0 and class weights inversely proportional to class frequency (HB and SV are 18% and 7% of the data), trained with the multinomial log-loss. Its four probabilities are inputs of the boosted heads; for the tournament charts it was trained on, its out-of-fold probabilities are used.

5.3 Boosted heads

Each task (category or tag) has independent binary heads: boosted classifiers with 200 iterations, learning rate 0.05, 15 leaves, at least 10 samples per leaf and balanced class weights, trained only on charts with a known label. The base input is the concatenation of the statistics, all rule scores and the four slot probabilities, so a head can start from the rule and from the slot model's notion of "this is an LN chart". The *boosted component* of the ensemble averages five heads per task, which differ in inputs and labels: base, base + windows, and base + windows + transitions on strict labels, and base + windows and base + windows + transitions on strict and relaxed labels (relaxed pairs at weight $\frac{1}{2}$). For the five SV heads (Reading, Memo and the three `sliders/*` tags) the boosted component is instead a single relaxed-label head on base + windows + transitions + reading + MinaCalc features. The reading features describe scroll speed only, and for the other tasks they would add 37 inputs that carry no signal; we use them where they apply (Section 7.4).

5.4 Per-fold teachers and targets

The neural models learn from the unlabelled majority of the corpus through *teachers*, models of the previous family whose predictions serve as soft targets where a label is missing (Figure 5). For fold k, every teacher is trained on folds $\neq k$ and predicts every chart, so its predictions are out-of-sample for fold k, and the students of fold k read them only on non-test charts. The chain is:

1. two boosted teachers: the mean of the relaxed-label heads on base + windows and on base + windows +

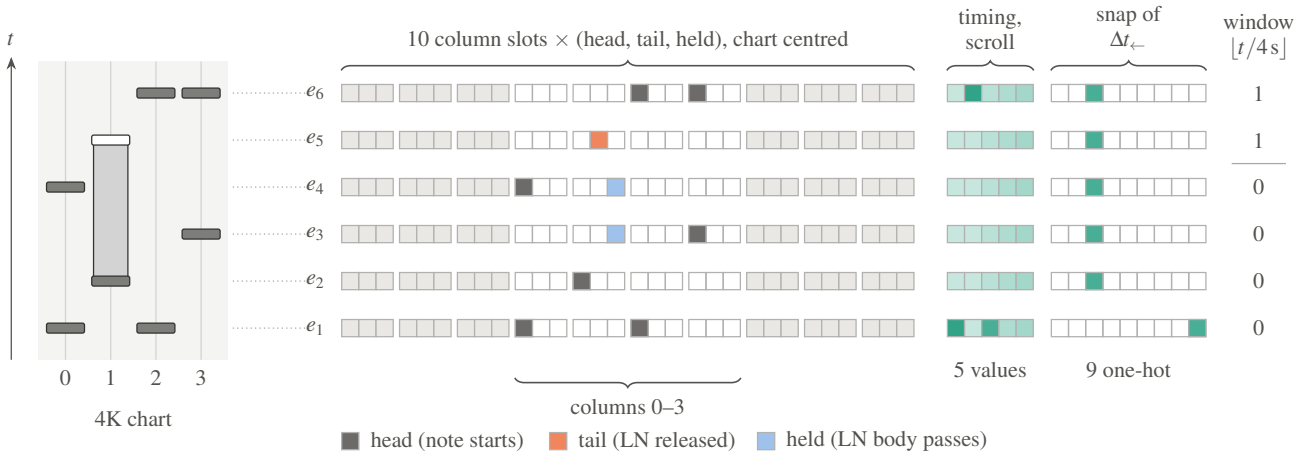


Figure 4: Event tokens. Each moment at which a note starts or a long note ends becomes one event (left: a 4K chart with one long note in column 1). Its vector holds head, tail and held bits for 10 column slots in which the chart's columns are centred (padding in grey), five timing and scroll values (shades) and the snap of the gap to the previous event. The window index groups events into 4-second windows for the encoder of Figure 6.

transitions, and the relaxed-label head with reading and MinaCalc features;

2. the MLP student (Section 5.5), distilled from the mean of the two boosted teachers;
3. the deep teacher T_k, the mean of the two boosted teachers and the MLP student, from which the event encoders are distilled.

For every chart and task, a student's target is the first that exists of the strict label (weight 1), the relaxed label (weight $\frac{1}{2}$) and the teacher's probability (weight 1). The students' binary losses are weighted cross-entropies with a positive weight $\min(10, \max(1, \sqrt{n_-/n_+}))$ per task, from the strict training labels.

5.5 MLP student

One multi-task network predicts all 31 tasks from the boosted heads' feature vector without rule scores and slot probabilities: statistics and window statistics (signed log), row transitions, reading features and MinaCalc ratings, z-scored on the fold's training charts. It has two hidden layers of 512 and 256 GELU units, dropout 0.1 on the input and 0.3 on the hidden layers, and is trained with AdamW [25] (learning rate 10^{-3}, weight decay 0.05, batches of 256) on every non-test chart with the targets of Section 5.4. Training stops when the mean AP on the inner-validation songs (over tasks with at least five validation positives) has not improved for 8 epochs, and the best epoch is kept. The component averages five seeds.

5.6 Deep event encoder

Architecture. The encoder reads the event tokens of Section 4.7 hierarchically (Figure 6). A linear layer maps each event to 64 channels and adds an embedding of the key count. Six residual blocks, each a GroupNorm [36], a convolution of width 3 with dilation 2^i ($i = 0, \ldots, 5$) and a gated linear unit [5], give every event a context of 127 events, enough for the patterns of Figure 2. Events are then pooled into 4-second windows by concatenating their mean and maximum, followed by a linear layer, GELU and LayerNorm. A learned position embedding is added to the window tokens, and a two-layer pre-norm transformer [34] with 4 heads relates the windows of the whole chart. Each task τ reads the window embeddings h_w with its own gated attention [20],

$$a_{\tau w} = \operatorname*{softmax}_w u_\tau^\top \big(\tanh(V_\tau h_w) \odot \sigma(U_\tau h_w) \big), \tag{3}$$

$$z_\tau = \textstyle\sum_w a_{\tau w} h_w, \tag{4}$$

and a linear classifier on z_τ gives the logit. The attention weights say in which windows a skillset was found. A four-way slot head reads the mean window embedding.

Stage 1: pretraining and distillation. On every non-test, non-validation chart of the fold (about 26 000), the encoder is trained for up to 30 epochs on the sum of four objectives:

- *masked events*: 15% of the events are replaced by a learned vector after the input layer, and from their encoded features the model predicts their head and tail bits per column and their log gap to the previous event (pattern grammar);
- *masked windows*: 15% of the window tokens are replaced by a learned vector before the transformer, and the model predicts 17 summary statistics of each masked window (log counts of events and notes; shares of chords, jacks, releases and held columns; mean gap and scroll speed; snap distribution) from the surrounding windows (structure);
- *statistics*: from the mean and maximum of the window embeddings, the model regresses the 260 chart and window statistics (signed log, z-scored, smooth ℓ_1), which are free targets on every chart;
- *tasks*: the weighted cross-entropy against the targets of Section 5.4, plus half the cross-entropy of the slot head on tournament charts.

Optimisation uses AdamW with learning rate $2 \cdot 10^{-3}$, weight decay 0.02, linear warm-up over the first 1/30 of the steps and cosine decay, gradient clipping at 1, and batches of charts of similar length with at most 64 000 padded events. Training

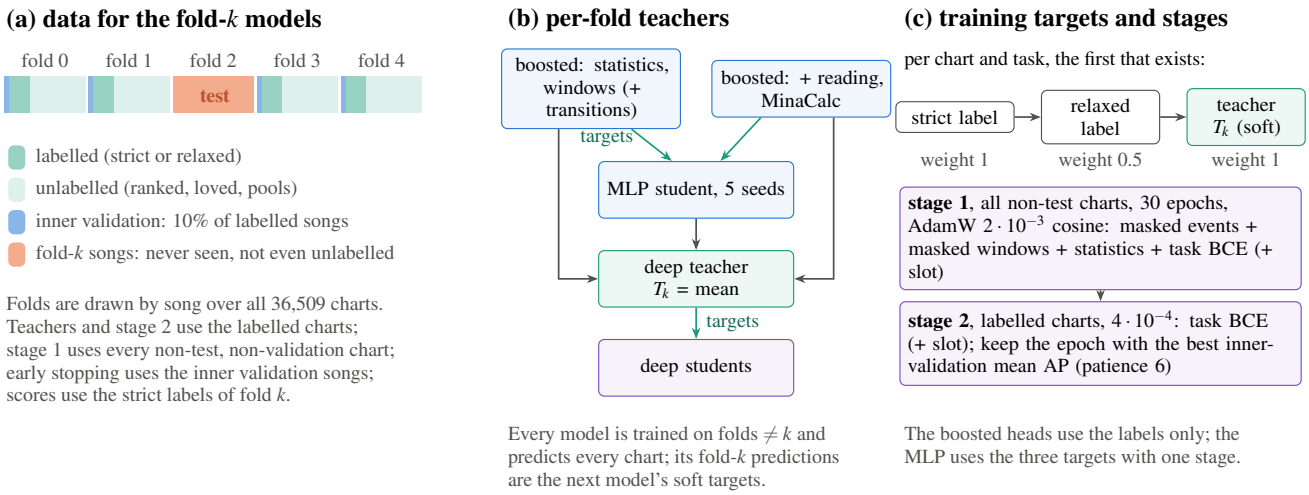


Figure 5: Training protocol for fold k. (a) Folds are drawn by song over the whole corpus, so fold-k songs are excluded from every stage of every fold-k model, including pretraining on unlabelled charts. (b) Each model family is distilled from the per-fold predictions of the previous ones. (c) Targets fall back from strict labels to relaxed labels to the teacher; the event encoders are trained in two stages.

charts are cropped to a random span of 4 000 events and mirrored with probability $\frac{1}{2}$.

Stage 2: fine-tuning. The self-supervised and statistics losses are dropped, and the model is fine-tuned on the charts that have a strict or relaxed label (teacher targets fill their unknown tasks) at learning rate $4 \cdot 10^{-4}$ for up to 25 epochs. After each epoch of both stages the mean AP on the inner-validation songs is computed; a stage stops after 6 epochs without improvement and keeps its best state. Test charts are read whole (up to 12 000 events), and the prediction is the mean over the chart and its mirror image.

Wide path. The second deep component adds the boosted heads' feature vector (statistics, window statistics, transitions, reading and MinaCalc features, z-scored) through an MLP ($128 \to 64 \to 35$ units) whose logits are added to the attention heads' task and slot logits, as in wide-and-deep models [3]. During training the whole wide path is switched off for a random half of the charts in each batch, so that the event encoder still has to classify on its own and the two paths do not co-adapt. The plain component averages 3 seeds and the wide component 6.

5.7 Ensemble

The final prediction for each task is the unweighted mean of the probabilities of the four components (boosted, MLP, deep, wide & deep), each itself the mean of its runs. Nothing in the combination is fitted, so the combination cannot overfit the evaluation folds.

5.8 Reported output

For a new chart the system reports its argmax slot, the categories of that slot whose score exceeds the head's threshold (for HB, the Rice and LN categories), any category of another slot whose score exceeds $\max(\text{threshold}, 0.8)$, Mixed Rice by the rule of Section 3.3, a TB flag, and timestamps of the densest jack, stream, LN and SV sections. Thresholds maximise F1 on the out-of-fold predictions.

6 Experimental setup

Slots. The slot model is evaluated by 5-fold stratified cross-validation grouped by song on the 4 029 slot-labelled charts. We report accuracy and macro-F1 with a cluster bootstrap over songs [11] (1 000 replicates).

Categories and tags. All models use the five song folds of Section 3.4. Inside the training folds, 10% of the songs that have a labelled chart are held out as *inner validation* for early stopping; the test fold is never used for any choice within a run. Each task is scored by average precision (AP) on the pooled out-of-fold predictions of its strictly labelled charts, which is threshold-free and suited to rare positives [6, 32], and each benchmark by the mean AP over its tasks (11 categories, 20 tags). The reference is the boosted head on base + windows with strict labels (*gbm+win*), the strongest single head of the boosted family. Differences are given with a paired cluster bootstrap over songs: songs are resampled with replacement 500 times, and on each resample the AP of every task is recomputed for both models on the same charts. The neural models were trained on one RTX 2060 SUPER (8 GB), about 1.5 hours for the five folds of one deep run; the boosted heads on four CPU cores.

7 Results

7.1 Tournament slots

The learned slot classifier reaches 98.1% accuracy and 0.978 macro-F1, against 88.5% and 0.844 for the slotting rules; the 9.6-point gain has a 95% interval of $[8.6, 10.6]$, and the improvement holds for each key mode (Table 2). Figure 7 locates the difference. Rice and scroll-speed charts are easy for both methods. The rules mislabel 427 of 738 hybrids (58%) as long-note charts, because tournament hybrids are far more LN-heavy than the rule's notion of "some LNs": their median LN share is 0.38, and a third of them exceed the point (0.37) at which the rule prefers LN (Figure 8). The learned model raises hybrid F1 from 0.566 to 0.951. Its 77 remaining errors

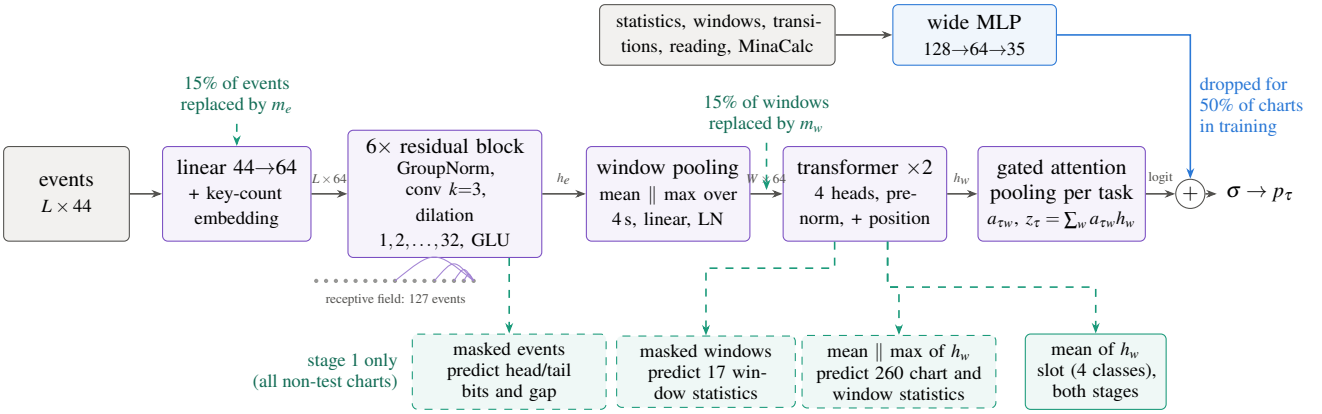


Figure 6: The event encoder. Dilated gated convolutions read local patterns from the event tokens, events are pooled into 4-second windows, a transformer relates the windows of the whole chart, and one gated attention pool per task selects the windows that carry the skillset. The wide path (blue) adds a logit from the chart-level features. The dashed heads are trained only in stage 1, on every non-test chart: masked event and masked window prediction, and regression of the chart and window statistics.

Table 2: Slot classification on 4,029 tournament charts (5-fold CV grouped by song). Brackets: 95% cluster-bootstrap intervals over songs. The accuracy gain is 9.6 points [8.6, 10.6].

			F1 per slot				accuracy by keys	
Method	Accuracy (%)	Macro-F1	RC	LN	HB	SV	4K	7K
Rule baseline	88.5 [87.5, 89.4]	0.844 [0.831, 0.856]	0.995	0.843	0.566	0.970	88.6	89.0
Gradient boosting	98.1 [97.7, 98.5]	0.978 [0.973, 0.983]	0.994	0.977	0.951	0.991	98.1	98.1

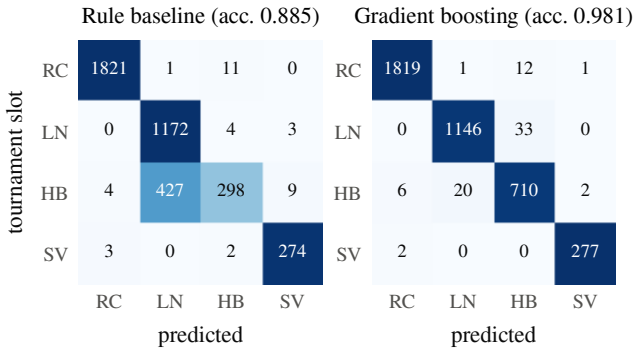


Figure 7: Slot confusion matrices (counts; shading is row-normalised). Nearly all rule errors are hybrids labelled LN.

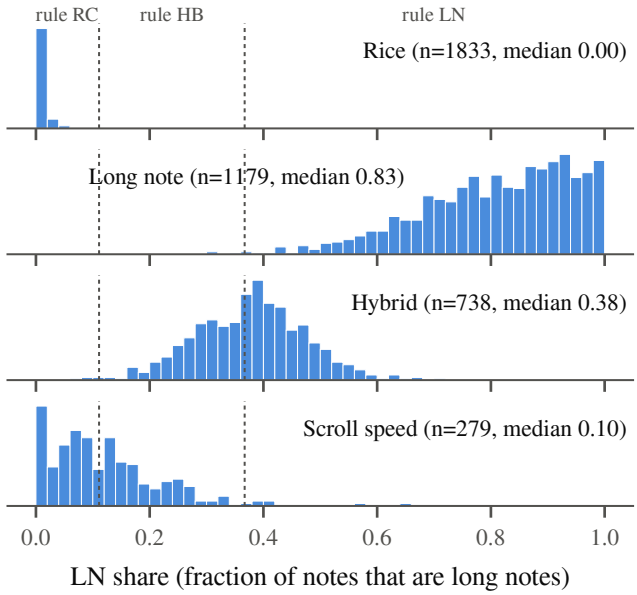


Figure 8: LN share of tournament charts by slot. Dashed lines mark where the rule's RC/HB and HB/LN scores cross. Hybrids span 0.1 to 0.7 and overlap the LN distribution.

are mostly the same boundary in both directions (median LN share 0.53) and cluster in a few tournaments, partly because the normaliser labels every o!mLN chart LN although some of its pools contain hybrids. Two quantities nearly determine the slot: LN share alone gives 90.0% accuracy, and adding the nine scroll-speed statistics gives 96.7%. A random forest or a logistic regression on the same 71 statistics comes within 0.2 points of boosting, so the gain comes from placing the boundaries where organisers place them rather than from the model class.

7.2 Categories and user tags

Tables 3 and 4 give the AP of every task, and Figure 9 shows them against the rules and the reference. The ensemble reaches a mean AP of 0.768 on the categories and 0.636 on the tags, against 0.719 and 0.588 for gbm+win and 0.564 and 0.452 for the rules. The gain over gbm+win is +0.049 on the categories, with a 95% song-bootstrap interval of $[+0.036, +0.060]$, and +0.047 on the tags, $[+0.040, +0.055]$. The ensemble is better than gbm+win on 11 of 11 categories and 20 of 20 tags.

The gains are largest where the reference lacked information. Memo rises from 0.27 to 0.53 and `sliders/low sv` from 0.53 to 0.66, through the reading features (Section 7.4). Skillsets defined by transitions or by sections gain 0.04 to 0.08: LN inverse (+0.08); generic hybrid, longjack, speedjack, tech, stream speed and handstream (+0.04 to +0.06); and Streams, Technical and Density/Inverse among the cate-

Table 3: Reviewer categories. Average precision on the strict out-of-fold labels (song-grouped 5-fold CV over all charts). pos./known: positive and labelled charts. gbm+win is the reference boosted head; the four components are averaged into the ensemble. Bold: best learned model. † SV heads, for which the boosted component uses the reading and MinaCalc features.

					components				
Task	pos.	known	rule	gbm+win	boosted	MLP	deep	wide & deep	ensemble
Chordjack	1057	3495	0.772	0.841	0.851	0.843	0.844	0.852	**0.856**
Streams	958	3327	0.479	0.585	0.610	0.624	0.626	0.633	**0.635**
Jumpstream/Handstream	1460	3612	0.679	0.817	0.827	0.837	0.839	**0.844**	0.843
Technical	723	3359	0.607	0.737	0.752	0.766	0.764	0.772	**0.776**
Stamina	1229	3384	0.614	0.779	0.785	0.785	0.782	0.790	**0.793**
Release	342	3349	0.368	0.718	0.732	0.717	0.740	0.733	**0.741**
Coordination	744	3556	0.543	0.752	0.765	0.772	0.777	0.772	**0.780**
Density/Inverse	486	3376	0.657	0.735	0.756	0.747	0.770	0.769	**0.773**
Reading†	485	3449	0.798	0.895	**0.915**	0.905	0.874	0.902	0.914
Memo†	43	3241	0.289	0.267	0.438	0.480	0.497	0.528	**0.531**
TB	585	7644	0.402	0.787	0.803	0.737	0.750	0.782	**0.805**
Mean			0.564	0.719	0.749	0.747	0.751	0.762	**0.768**

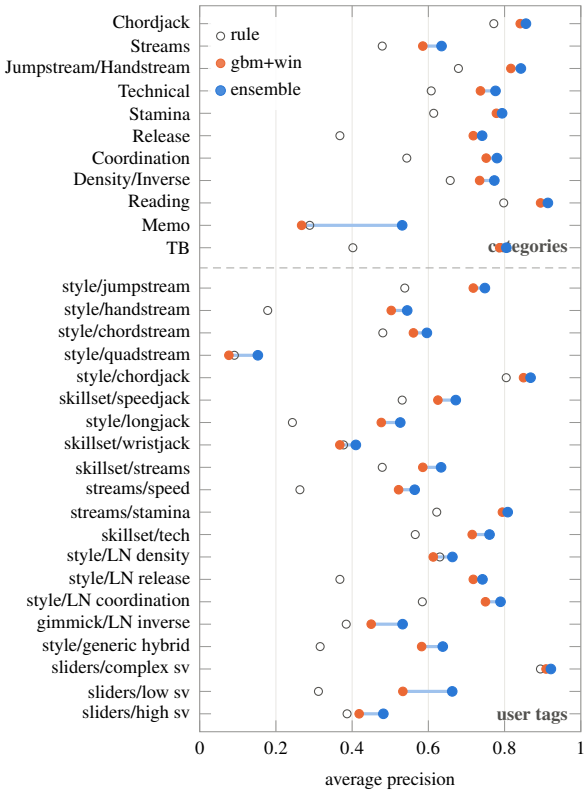


Figure 9: Average precision per task: rule, the boosted reference gbm+win, and the ensemble (lines join the last two).

gories (+0.04 to +0.05). Skillsets that one statistic already captures, such as chordjack (0.87) or complex SV (0.92), gain least. The weakest tasks are the rarest: `style/quadstream` has 36 positives and stays at 0.15 AP, and wristjack reaches 0.41.

7.3 What each component adds

Each of the four components alone beats the reference by 0.03 to 0.04 on both benchmarks (Table 5). The wide & deep encoder is the strongest single model (0.762 / 0.629); the MLP, which sees the same features as the boosted heads, matches the plain encoder on the tags. The plain mean of the four is better than any of them, by about 0.006 on the categories and 0.007 on the tags, and removing any one component costs 0.001 to 0.003 on at least one benchmark. The families make different errors: trees split on single statistics, the MLP combines all features smoothly and learns from unlabelled charts through the teacher, and the encoders see the order of events, which no chart-level feature fully describes. The neural students also see far more charts: every non-test chart enters their training through the teacher, while a boosted head sees only the 3 200 to 7 600 charts labelled for its task.

7.4 Reading features and the SV heads

The SV categories depend on what the player sees. On Memo, the boosted component with the reading features reaches 0.44 AP against 0.27 without them, and the ensemble 0.53 against 0.41; over the five SV heads this is the difference between 0.768 and 0.756 on the category mean (Table 5, last row). Reading is already well predicted by the scroll-speed statistics (0.89 for gbm+win) and changes little. Memo remains the noisiest category: it has 43 positives, 26 of them tournament charts, and the AP of single wide & deep runs ranges from 0.45 to 0.54 across seeds (quadstream: 0.10 to 0.19), a spread of about ±0.004 on a benchmark mean.

8 Discussion and limitations

Labels limit the system. The least accurate and least stable tasks are those with the fewest labels: Memo, quadstream, high and low SV and LN inverse each have fewer than 250 strict positives. More votes or sub-slot labels for these skillsets are the most direct improvement, more than further model capacity: the four components differ in architecture far more than in accuracy.

Crowd labels. User tags are a noisy proxy for what a pooler means by a skillset. Voters see a chart's existing top tags, so votes may herd; popular charts are over-represented among the strict negatives, which require eight votes; and some tags carry community-specific meanings, as `skillset/reading`

Table 4: User tags. Average precision on the strict out-of-fold labels (song-grouped 5-fold CV over all charts). pos./known: positive and labelled charts. gbm+win is the reference boosted head; the four components are averaged into the ensemble. Bold: best learned model. † SV heads, for which the boosted component uses the reading and MinaCalc features.

					components				
Task	pos.	known	rule	gbm+win	boosted	MLP	deep	wide & deep	ensemble
`style/jumpstream`	945	3276	0.538	0.718	0.736	0.735	0.742	0.747	**0.748**
`style/handstream`	298	3199	0.179	0.503	0.521	0.534	0.529	0.536	**0.545**
`style/chordstream`	479	3324	0.481	0.561	0.581	0.592	0.586	0.591	**0.596**
`style/quadstream`	36	3218	0.091	0.077	0.115	0.147	**0.164**	0.158	0.153
`style/chordjack`	1020	3420	0.804	0.850	0.860	0.863	0.867	0.865	**0.868**
`skillset/speedjack`	743	3297	0.531	0.625	0.651	0.632	0.657	0.661	**0.672**
`style/longjack`	214	3211	0.243	0.477	0.494	0.502	0.495	0.523	**0.526**
`skillset/wristjack`	310	3200	0.377	0.368	0.385	0.408	0.401	0.404	**0.410**
`skillset/streams`	958	3327	0.479	0.585	0.610	0.624	0.623	0.631	**0.633**
`streams/speed`	675	3261	0.263	0.522	0.547	0.549	0.542	0.554	**0.564**
`streams/stamina`	1228	3294	0.622	0.794	0.802	0.800	0.797	0.802	**0.808**
`skillset/tech`	684	3352	0.566	0.715	0.732	0.749	0.748	0.757	**0.760**
`style/LN density`	324	3269	0.630	0.613	0.639	0.654	**0.665**	**0.665**	0.663
`style/LN release`	342	3349	0.368	0.718	0.732	0.717	**0.744**	0.733	0.742
`style/LN coordination`	722	3489	0.584	0.750	0.769	0.781	**0.790**	0.783	0.789
`gimmick/LN inverse`	209	3247	0.384	0.450	0.490	0.530	**0.541**	0.534	0.532
`style/generic hybrid`	421	3348	0.316	0.582	0.616	0.627	0.625	0.622	**0.638**
`sliders/complex sv`†	247	3304	0.894	0.909	0.919	**0.925**	0.900	0.907	0.921
`sliders/low sv`†	183	3268	0.312	0.533	0.643	0.641	0.610	0.643	**0.663**
`sliders/high sv`†	141	3256	0.387	0.418	0.470	**0.482**	0.468	0.457	**0.482**
Mean			0.452	0.588	0.616	0.625	0.625	0.629	**0.636**

Table 5: Mean AP of each component alone and of the ensemble without it. The four components are averaged with equal weights; nothing is fitted on the evaluation folds.

Model	Categories	Tags
gbm+win (reference)	0.719	0.588
boosted alone	0.749	0.616
MLP alone	0.747	0.625
deep alone	0.751	0.625
wide & deep alone	0.762	0.629
ensemble of the four	0.768	0.636
without boosted	0.765	0.636
without MLP	0.767	0.634
without deep	0.766	0.633
without wide & deep	0.766	0.635
SV heads: boosted without reading	0.756	0.635

shows. Relaxed labels use partially voted charts at half weight; an estimate of the labelling propensity [2, 12] or a model of voter agreement could use them better.

Choices made on the evaluation folds. The ensemble has no fitted weights, but two design choices were made while looking at cross-validated scores: which feature families the components use, and the reading-feature boosted head for the five SV heads. Both are coarse (one decision for a family of heads, not per task), but the reported means are slightly optimistic for a new sample of charts.

Domain shift. The heads are trained mostly on ranked charts and applied to unranked requests. Tournament customs are closer to requests than ranked charts are, and the category labels include them, but we have no labelled sample of requests themselves.

Cost. The deep components need a GPU to train but not to predict. The MLP student alone reaches 0.747 / 0.625, most of the ensemble's gain over the reference, and runs on a CPU in milliseconds; it is the natural model when the ensemble is too heavy for a service.

9 Conclusion

osu!mania charts can be described automatically in the vocabulary reviewers and tournament organisers use, from public data alone. Official mappools provide expert slot labels, and a boosted model on 71 interpretable statistics reproduces them with 98.1% accuracy under song-grouped cross-validation, against 88.5% for slotting rules. Crowd-voted user tags, read as positive–unlabelled data with strict and relaxed thresholds, supervise 11 reviewer categories and 20 tags. An ensemble of boosted heads, a distilled MLP and two event encoders pretrained on the unlabelled corpus reaches a mean AP of 0.768 and 0.636, $+0.049$ and $+0.047$ above the best boosted head, without any model seeing a test song. Window-level and transition-level descriptions of the chart, and reading features computed from the scroll position, make section-level and scroll-speed skillsets learnable. The encoders' attention weights already locate skillsets in time; evaluating those timestamps, and evaluating on labelled ranking requests, are the next steps.

Acknowledgements. We thank the organisers of the osu!mania tournaments whose mappools are documented

on the osu! wiki, the players who vote user tags, and the Etterna developers for MinaCalc.

References

[1] Sojeong An, Nathan Pruyne, Chris Donahue, and Dasaem Jeong. Transfer learning for rhythm game chart generation from music transcription and across games. *arXiv preprint arXiv:2610.14955*, 2026.

[2] Jessa Bekker and Jesse Davis. Learning from positive and unlabeled data: A survey. *Machine Learning*, 109 (4):719–760, 2020.

[3] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, pages 7–10, 2016.

[4] Keunwoo Choi, György Fazekas, and Mark Sandler. Automatic tagging using deep convolutional neural networks. In *Proceedings of the 17th International Society for Music Information Retrieval Conference (ISMIR)*, pages 805–811, 2016.

[5] Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 933–941, 2017.

[6] Jesse Davis and Mark Goadrich. The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240, 2006.

[7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[8] Thomas G. Dietterich. Ensemble methods in machine learning. In *Multiple Classifier Systems*, volume 1857 of *Lecture Notes in Computer Science*, pages 1–15, 2000.

[9] Thomas G. Dietterich, Richard H. Lathrop, and Tomás Lozano-Pérez. Solving the multiple instance problem with axis-parallel rectangles. *Artificial Intelligence*, 89 (1–2):31–71, 1997.

[10] Chris Donahue, Zachary C. Lipton, and Julian McAuley. Dance dance convolution. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1039–1048, 2017.

[11] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1993.

[12] Charles Elkan and Keith Noto. Learning classifiers from only positive and unlabeled data. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 213–220, 2008.

[13] Etterna contributors. MinaCalc: the Etterna difficulty calculator. `https://github.com/etternagame/etterna`, 2024. Per-skillset chart ratings (stream, jumpstream, handstream, stamina, jackspeed, chordjack, technical).

[14] Billy Joe Franks, Benjamin Dinkelmann, Sophie Fellenz, and Marius Kloft. Ordinal regression for difficulty estimation of StepMania levels. *arXiv preprint arXiv:2301.09485*, 2023.

[15] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29 (5):1189–1232, 2001.

[16] Scott A. Golder and Bernardo A. Huberman. Usage patterns of collaborative tagging systems. *Journal of Information Science*, 32(2):198–208, 2006.

[17] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35, Datasets and Benchmarks Track*, 2022.

[18] Emily Halina and Matthew Guzdial. TaikoNation: Patterning-focused chart generation for rhythm action games. In *Proceedings of the 12th Workshop on Procedural Content Generation (PCG)*, 2021. arXiv:2107.12506.

[19] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[20] Maximilian Ilse, Jakub M. Tomczak, and Max Welling. Attention-based deep multiple instance learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 2127–2136, 2018.

[21] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):15:1–15:21, 2012.

[22] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 3146–3154, 2017.

[23] Paul Lamere. Social tagging and music information retrieval. *Journal of New Music Research*, 37(2):101–114, 2008.

[24] Ziyun Liu and Chris Donahue. Osu2MIR: Beat tracking dataset derived from osu! data. *arXiv preprint arXiv:2509.12667*, 2025.

[25] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[26] osu-finder contributors. osu-finder (v1.3.0). `https://pypi.org/project/osu-finder/1.3.0/`, 2024. Python package; jump/stream classification from strain ratios.

[27] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[28] Raregendary. SimpleBeatmapAnalyzer. `https://github.com/Raregendary/SimpleBeatmapAnalyzer`, 2024. Rule-based osu!standard stream and jump statistics.

[29] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282, 2017.

[30] David R. Roberts, Volker Bahn, Simone Ciuti, Mark S. Boyce, Jane Elith, Gurutzeta Guillera-Arroita, Severin Hauenstein, José J. Lahoz-Monfort, Boris Schröder, Wilfried Thuiller, David I. Warton, Brendan A. Wintle, Florian Hartig, and Carsten F. Dormann. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8): 913–929, 2017.

[31] rosu-pattern-detector contributors. rosu-pattern-detector (v0.1.1). `https://docs.rs/crate/rosu-pattern-detector/0.1.1`, 2025. Rust crate; rule-based osu!mania pattern detection.

[32] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3):e0118432, 2015.

[33] Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.

[34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, pages 5998–6008, 2017.

[35] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

[36] Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 3–19, 2018.

[37] Qizhe Xie, Minh-Thang Luong, Eduard Hovy, and Quoc V. Le. Self-training with Noisy Student improves ImageNet classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10687–10698, 2020.

[38] Jayeon Yi, Sungho Lee, and Kyogu Lee. Beat-aligned spectrogram-to-sequence generation of rhythm-game charts. *arXiv preprint arXiv:2311.13687*, 2023.

[39] Hsiang-Fu Yu, Prateek Jain, Purushottam Kar, and Inderjit S. Dhillon. Large-scale multi-label learning with missing labels. In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, pages 593–601, 2014.

A Chart statistics

Table 6: The 71 chart statistics by family, as named in the released code.

Family	Features
Meta (5)	`keys`, `od`, `hp`, `notes`, `duration_s`
Density (10)	`nps`, `nps_per_key`, `nps_p50`, `nps_p90`, `nps_max`, `nps_peak_ratio`, `stamina`, `density_cv`, `breaks`, `rows_ps`
Chords/jacks/streams (22)	`notes_per_row`, `row_single`, `row_jump`, `row_hand`, `row_quad`, `row_chord_frac_of_keys`, `row_full`, `jack_row`, `jack_note`, `chordjack_row`, `full_jack_row`, `stream_row`, `jumpstream_row`, `handstream_row`, `jack_gap_ms`, `row_gap_ms_p10`, `row_gap_ms_p50`, `jack_chain3`, `jack_longest`, `trill_row`, `col_imbalance`, `anchor`
Rhythm (14)	`snap_1/2`, `snap_1/4`, `snap_1/8`, `snap_1/16`, `snap_1/3`, `snap_1/6`, `snap_1/12`, `snap_other`, `snap_triplet`, `snap_fine`, `rhythm_variety`, `gap_ratio_change`, `bpm_main`, `bpm_changes`
Long notes (11)	`ln_ratio`, `ln_count_ps`, `ln_len_ms_p50`, `ln_len_beats_p50`, `ln_short`, `ln_long`, `ln_hold_cov`, `ln_inverse`, `ln_release_offgrid`, `ln_held_mean`, `has_ln`
Scroll speed (9)	`sv_points`, `sv_points_ps`, `sv_log_std`, `sv_off_frac`, `sv_far_frac`, `sv_stop_frac`, `sv_fast_frac`, `sv_changes_ps`, `sv_changes_near_notes_ps`

B Reading features

Table 7: The 22 reading features (Section 4.5). $v = (T - o(T))/H$ is a row's visible time in units of $H = 600$ ms; $r = \log(\Delta x/\Delta t)$ the log ratio of on-screen to time gap between consecutive rows.

Features	Definition
`rd_vis_p5/p25/p50/p95`	percentiles of v
`rd_vis_short/sudden/long`	share of rows with $v < 0.5$, $v < 0.25$, $v > 2$
`rd_vis_logstd`	std of $\log v$
`rd_gap_logstd/p5/p95`	std and percentiles of r
`rd_stacked/spread`	share of row pairs with $\Delta x/\Delta t < 0.2$, > 3
`rd_gap_jitter`	mean $\lvert r_{j+1} - r_j \rvert$
`rd_hit_speed_std`	std of $\log s$ at hit times
`rd_hit_speed_local_std`	the same within 2 s blocks, averaged
`rd_moving_changes_ps/frac`	speed changes ($\lvert\Delta \log s\rvert > 0.05$) while the next row is visible, per second and as a share of changes
`rd_teleports_pm`	segments with $s > 4$, < 80 ms, while a row is visible, per minute
`rd_freezes_pm`	segments with $s < 0.1$, > 150 ms, while a row is visible, per minute
`rd_crowd_ratio_p5/p95`	rows on screen at once relative to constant scroll, percentiles

C Hyperparameters

Table 8: Hyperparameters of the category and tag models.

Boosted heads (scikit-learn `HistGradientBoostingClassifier`)	
trees	200 iterations, learning rate 0.05, 15 leaves, ≥ 10 samples per leaf, ℓ_2 1.0, balanced class weights
labels	strict; or strict + relaxed at weight 0.5
MLP student	
network	input dropout 0.1, $512 \to 256$ GELU, dropout 0.3
training	AdamW, lr 10^{-3}, weight decay 0.05, batch 256, ≤ 80 epochs, patience 8, 5 seeds
Event encoder	
tokens	44 values per event, events merged within 2 ms, windows of 4 s, ≤ 256 windows
convolutions	64 channels, 6 GLU blocks, kernel 3, dilations $1 \ldots 32$, GroupNorm, dropout 0.1
transformer	2 pre-norm layers, 4 heads, feed-forward 128, dropout 0.1
MIL heads	gated attention, 32 attention units per task
wide path	dropout 0.1, $128 \to 64 \to 35$ GELU, dropped for 50% of charts
stage 1	≤ 30 epochs, AdamW lr $2 \cdot 10^{-3}$, weight decay 0.02, 15% events and 15% windows masked, all losses weight 1, slot loss 0.5
stage 2	≤ 25 epochs, lr $4 \cdot 10^{-4}$
both stages	warm-up 1/30 of steps then cosine, gradient clip 1, $\leq 64\,000$ padded events per batch, random crops of 4 000 events, mirror $p = 0.5$, early stopping patience 6 on inner-validation mean AP
test	whole chart ($\leq 12\,000$ events), mean with the mirror image
Folds	
	5 folds grouped by song over 36,509 charts; 10% of labelled training songs for inner validation